FRED TALKS

24th April 2026

CONTENTS

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · SOURCE

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the M×N integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it’s thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The MCP SDKs recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including Claude Cowork, Claude Managed Agents, and channels in Claude Code.

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our directory, used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single create_issue_from_thread tool beats get_thread + parse_messages + create_issue + link_attachment. See writing effective tools for agents to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. Cloudflare's MCP server is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

MCP Apps is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

Elicitation lets your server pause mid-tool call to ask the user for input. Form mode sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. URL mode hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest MCP spec supports CIMD (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. Vaults in Claude Managed Agents covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents (*clients*) connect to and work with tools and data sources they need (*servers*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

Tool search defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our testing, tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: advanced tool use

Process tool results in code with programmatic tool calling

Programmatic tool calling processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly 37% on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See *advanced tool use* for the full breakdown.

Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our data plugin for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: Extending Claude's capabilities with skills and MCP servers

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. Canva, Notion, Sentry, and more do this today in Claude, listing the skill next to their connector in our web directory.

To make that pairing portable across every client, the MCP community is actively working on an extension for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

DeepSeek V4 - almost on the frontier, a fraction of the price

SIMON WILLISON'S WEBLOG: ENTRIES · 24 APR 2026 · SOURCE

DeepSeek V4—almost on the frontier, a fraction of the price

Chinese AI lab DeepSeek's last model release was V3.2 (and V3.2 Speciale) last December. They just dropped the first of their hotly anticipated V4 series in the shape of two preview models, DeepSeek-V4-Pro and DeepSeek-V4-Flash.

Both models are 1 million token context Mixture of Experts. Pro is 1.6T total parameters, 49B active. Flash is 284B total, 13B active. They're using the standard MIT license.

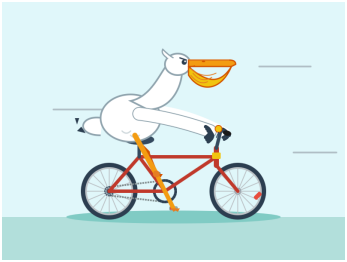
I think this makes DeepSeek-V4-Pro the new largest open weights model. It's larger than Kimi K2.6 (1.1T) and GLM-5.1 (754B) and more than twice the size of DeepSeek V3.2 (685B).

Pro is 865GB on Hugging Face, Flash is 160GB. I'm hoping that a lightly quantized Flash will run on my 128GB M5 MacBook Pro. It's *possible* the Pro model may run on it if I can stream just the necessary active experts from disk.

For the moment I tried the models out via OpenRouter, using llm-openrouter:

```
llm install llm-openrouter
llm openrouter refresh
llm -m openrouter/deepseek/deepseek-v4-pro 'Generate an SVG of a peli
```

Here's the pelican for DeepSeek-V4-Flash:



And for DeepSeek-V4-Pro:

For comparison, take a look at the pelicans I got from DeepSeek V3.2 in December, V3.1 in August, and V3-0324 in March 2025.

So the pelicans are pretty good, but what's really notable here is the *cost*. DeepSeek V4 is a very, very inexpensive model.

Here's DeepSeek's pricing page. They're charging $0.14/million tokens input and $0.28/million tokens output for Flash, and $1.74/million input and $3.48/million output for Pro.

Here's a comparison table with the frontier models from Gemini, OpenAI and Anthropic:

Model	Input ($/M)	Output ($/M)
DeepSeek V4 Flash	$0.14	$0.28
GPT-5.4 Nano	$0.20	$1.25
Gemini 3.1 Flash-Lite	$0.25	$1.50
Gemini 3 Flash Preview	$0.50	$3
GPT-5.4 Mini	$0.75	$4.50
Claude 4.5 Haiku	$1	$5
DeepSeek V4 Pro	$1.74	$3.48
Gemini 3.1 Pro	$2	$12
GPT-5.4	$2.50	$15
Claude Sonnet 4 / 4.5	$3	$15
Claude Opus 4.5	$5	$25
GPT-5.5	$5	$30

DeepSeek-V4-Flash is the cheapest of the small models, beating even OpenAI's GPT-5.4 Nano. DeepSeek-V4-Pro is the cheapest of the larger frontier models.

This note from the DeepSeek paper helps explain why they can price these models so low—they've focused a great deal on efficiency with this release, especially for longer context prompts:

> In the scenario of 1M-token context, even DeepSeek-V4-Pro, which has a larger number of activated parameters, attains only 27% of the single-token FLOPs (measured in equivalent FP8 FLOPs) and 10% of the KV cache size relative to DeepSeek-V3.2. Furthermore, DeepSeek-V4-Flash, with its smaller number of activated parameters, pushes efficiency even further: in the 1M-token context setting, it achieves only 10% of the single-token FLOPs and 7% of the KV cache size compared with DeepSeek-V3.2.

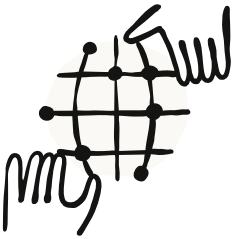
DeepSeek's self-reported benchmarks in their paper show their Pro model competitive with those other frontier models, albeit with this note:

> Through the expansion of reasoning tokens, DeepSeek-V4-Pro-Max demonstrates superior performance relative to GPT-5.2 and Gemini-3.0-Pro on standard reasoning benchmarks. Nevertheless, its performance falls marginally short of GPT-5.4 and Gemini-3.1-Pro, suggesting a developmental trajectory that trails state-of-the-art frontier models by approximately 3 to 6 months.

I'm keeping an eye on huggingface.co/unsloth/models as I expect the Unsloth team will have a set of quantized versions out pretty soon. It's going to be very interesting to see how well that Flash model runs on my own machine.

Anthropic and NEC collaborate to build Japan's largest AI engineering workforce

ANTHROPIC NEWS · 24 APR 2026 · SOURCE



NEC Corporation will use Claude as it builds one of Japan's largest AI-native engineering organizations, making it available to approximately 30,000 NEC Group employees worldwide.

As part of this strategic collaboration, NEC will become Anthropic's first Japan-based global partner. Together, we will develop secure, industry-specific AI products for the Japanese market, starting with tools for finance, manufacturing, and local government.

"This long-term partnership with Anthropic enables NEC to maximize the potential of AI in the Japanese market," said Toshifumi Yoshizaki, Executive Officer and COO of NEC Corporation. "Together, we aim to create solutions that meet the high safety, reliability, and quality standards demanded by companies and public administration in Japan."

NEC and Anthropic will jointly develop secure, domain-specific AI products for Japanese customers in sectors like finance, manufacturing, and cybersecurity.

In addition, NEC is already integrating Claude into its Security Operations Center services to help defend customers against increasingly sophisticated cybersecurity threats. Claude will also be integrated into the next-generation cybersecurity service NEC is currently providing.

Claude, including Claude Opus 4.7, and Claude Code will be incorporated into NEC BluStellar Scenario, a program that provides consulting, AI tools, security, and digital infrastructure to businesses, starting with its offerings for data-driven management and customer experience, and gradually expanding to others.

Internally, NEC will establish a Center of Excellence to develop a highly skilled, AI-enabled engineering organization, supported by technical enablement and training from Anthropic. NEC aims to build one of Japan's largest AI-native engineering teams, who will use Claude Code in their work.

As part of its long-running Client Zero initiative, in which NEC serves as its own first customer before offering its technology to clients, NEC will also expand its use of Claude Cowork across its internal business operations.

Claude is now being deployed to NEC Group employees around the world, and our joint development of industry-specific AI solutions is underway. Learn more about NEC's value-creation model at NEC BluStellar.

Claude, Claude Code, and Claude Cowork are Anthropic products. NEC BluStellar is an offering from NEC Corporation.

Equity for Europeans

ARMIN RONACHER · 23 APR 2026 · SOURCE

If you spend enough time in US business or finance conversations, one word keeps showing up: **equity**.

Coming from a German-speaking, central European background, I found it surprisingly hard to fully internalize what that word means. More than that, I find it very hard to talk to other Europeans about it. Worst of all it's almost impossible to explain it in German without either sounding overly technical or losing an important part of the meaning.

This post is in English, but it is written mostly for readers in Germany, Austria, and Switzerland, and more broadly for people from continental Europe. I move between "German-speaking" and "continental European" a bit. They are not the same thing, of course, but many continental

European countries share a civil-law background that differs sharply from the English common-law and equity tradition. The words differ by language and jurisdiction, but the conceptual gap I am interested in shows up in similar ways.

In US usage, the word “equity” appears everywhere:

- real estate: “build equity in your home”
- startups: “employees get equity”
- public markets: “equity investors”
- private deals: “take an equity stake”
- personal finance: “negative equity in a car”
- social policy: “diversity, equity, and inclusion”

If you try to translate this into German, you have to choose words. Of course we can say *Eigenkapital*, *Beteiligung*, *Anteil*, *Vermögen*, *Nettovermögen*, or sometimes *Substanzwert*. In narrow contexts, each can be correct, but none of them carries the full concept. I find that gap interesting, because language affects default behavior and how we think about things.

One Word, Shared Meanings

In American English, “equity” often carries multiple things at once. I believe the following ones to be the most important ones:

1. A legal-fairness dimension: historically tied to equity in law
2. A financial-accounting dimension: residual value after debt
3. A cultural dimension: ownership as a path to wealth and agency

If you open Wikipedia, you will find many more distinct meanings of equity, but they all relate to much the same concept, just from different angles.

German, on the other hand, can express each of these layers precisely, including the subtleties within each, but it uses different words and there is no common, everyday umbrella word that naturally bundles all three.

When a concept has one short, reusable, positive word, people can move it across contexts very easily. When the concept is split into technical fragments, it tends to stay technical, and people do not necessarily think of these things as related at all in a continental European context.

How Equity Got Here

What is hard for Europeans to understand is how the financial meaning of equity appeared, because it did not appear out of nowhere. The word's original meaning comes from fairness or impartiality, and it made it to modern English via Old French and Latin (*equité* / *aequitas*).

Historically, English law had separate traditions: common law courts and courts of equity (especially the Court of Chancery). Equity in law was about fairness, conscience, and remedies where strict common law rules were too rigid. Take mortgages for instance: in older English practice, a mortgage could transfer title as security. Under strict common law, missing a deadline could mean losing the property entirely. Courts of equity developed the "equity of redemption": a borrower could still redeem by paying what was owed.

That equitable interest became foundational for how ownership and claims were understood. In finance, equity came to mean not just a number, but a claim: the residual owner's stake after prior claims are satisfied.

The European Split

German and continental European legal development took a different path. Civil law systems did not build the same separate institutional track of "equity courts" versus common law courts. Fairness principles absolutely exist, but inside the codified system, not as a parallel jurisdiction with its own language and mythology.

As a result, German vocabulary has many different words, and they are highly domain-specific. There are equivalents in other languages, and to some degree they exist in English too:

- company balance sheet: *Eigenkapital*
- ownership share: *Beteiligung*, *Anteil*
- unrealized asset value: *stille Reserven*
- household wealth: *Vermögen*, *Nettovermögen*
- investment action: *Anlage*, *Investition*

- residual net assets: *Reinvermögen*

This precision is useful for legal drafting and accounting. But it also means we have less of the shared mental package that many Americans get from “equity”: own a piece, carry risk, participate in upside, build wealth.

Schuld Is Not Just Debt

There is another linguistic oddity worth noting: in German, “Schuld” can mean both debt/liability and guilt, and I think that too has changed how we think about equity.

“Schuld” in everyday language makes debt feel more morally charged than it does in the US. Indebtedness is often framed as a burden, and it is not thought of as a tool at all.

US financial language, by contrast, often frames debt more instrumentally and pairs it with an explicit positive counterpart: equity. Equity is what is yours after debt, what can appreciate, what can be transferred, and what can give you control.

In American financial language, debt is not as morally burdened, and equity is more than the absence of debt: it is the positive claim on the balance sheet — ownership, optionality, control, and upside.

Practical Matters

If you grew up with German-speaking framing, many US statements around equity can sound ideological or naive. From a continental European lens, they can sound like imported jargon or hollow. But if we ignore the concept, we lose something practical:

- We discuss salaries in cash terms but under-discuss ownership.
- We treat employee participation as exotic instead of normal.
- We under-explain compounding and intergenerational transfer.
- We miss a language for talking about agency through ownership.

I am not saying German-speaking Europeans are incapable of this mindset. Obviously we are not. But we clearly tend to think about these things differently.

Normalize Equity

When you hear “equity,” it helps to think of it as a rightful stake. Historically, it is connected to fairness and the recognition of a claim where strict rules would be too rigid. Financially, it is the part that remains after prior obligations. Culturally, it is something that can grow into control, agency, and upside.

That is not a perfect definition, but it captures why the term is so sticky in American discourse. It combines a present claim with a future possibility. It is not just what remains after debt; it is the part that can grow, compound, and give you agency.

If Europeans want to talk more seriously about entrepreneurship, retirement, housing, and wealth building, we would benefit from a stronger everyday vocabulary for exactly this idea. We need a longing for equity so that ownership does not remain something for founders, lawyers, accountants, and wealthy families, but becomes a normal part of how people think about work, risk, and their future.

Not because we should imitate America, but because this mental model helps people make clearer decisions about ownership, incentives, and long-term agency. For Europe, that shift feels long overdue.

Sign of the future: GPT-5.5

ETHAN MOLLICK · 23 APR 2026 · SOURCE

One impressive step on the curve

I had early access to GPT-5.5[1], and I think it is a big deal. It is a big deal because it indicates that we are not done with the rapid improvement in AI. It is also a big deal because it is just plain good. And it is a big deal because even with all of this, the frontier of AI ability remains jagged.

It is increasingly hard to quickly demonstrate each generational change as AI has gotten better, since a lot of the old things AI was bad at, like math or counting letters in words, are now trivial for AI to do. So, I will give you the complicated details, but first, a simple example that I think is a good illustration. What AI models are best at is coding, so I gave a coding challenge to AIs ranging from OpenAI’s first reasoning model, o3 (released a year and a week ago!) to the

current best open weights model (Kimi K2.6) to the new GPT-5.5 Pro: “build me a procedurally generated 3D simulation showing the evolution of a harbor town from 3000 BCE to 3000 AD, it should look beautiful and allow me to have some control over it.”

Then I posted every answer to this gallery so you can experiment with them (actually, I had GPT-5.5 Codex build the gallery page for me). You should play with them to feel the difference, but you can see a few of these examples below. In addition to being better along all the other dimensions, only GPT-5.5 Pro actually modelled an evolving town, rather than just generating new building replacements over time. GPT-5.5 Pro is also much faster than its previous iteration: GPT-5.4 Pro took 33 minutes to complete the task, GPT-5.5 Pro took 20.

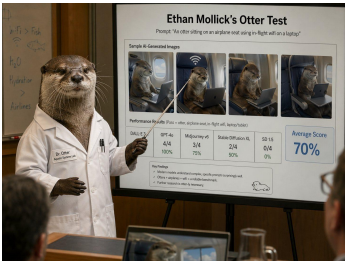
Some content could not be imported from the original document. View content ↗

Models, Apps, and Harnesses

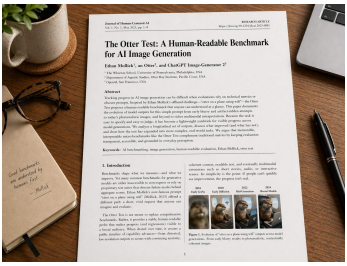
I have been encouraging you to think about AI not as a single thing, but as a set of three interlinked concepts. You need to consider **models**, like Opus 4.7, Gemini 3.1, or (now) GPT-5.5. You also want to pay attention to **apps**, which are the products you actually use to talk to a model, and which let models do real work for you. The most common app is the website for each of these models: chatgpt.com, claude.ai, gemini.google.com. But, increasingly, desktop applications like Claude Code, Claude Cowork, and OpenAI Codex are becoming the most useful apps for AI. Finally, there are **harnesses**, the tools that an AI can use and how the AI models are hooked up to these tools. Tools allow the AI to control your computer, write code, do research, and make images.

OpenAI has made advances in all three areas. On the model front, GPT-5.5 is a powerful family of models, with GPT-5.5 Pro (accessible only on the website) the most competent. There have also been major advances recently in apps, with OpenAI’s Codex increasingly following the path of the excellent Claude Code and making an accessible and useful desktop application. Finally, there are harnesses and the tools they can use. There have been a lot of new harness improvements, but one of the most interesting is from OpenAI, which has a new image model

This new model can now render high-quality text and create almost any picture you can describe. Long-time readers know about my Otter Test, which asks the AI to make an image of an otter on a plane using wifi. Rather than describe it again, let’s let the new image model (sometimes called GPT-imagegen-2) explain it for me: “a photo of an otter scientist demonstrating the results of Ethan Mollick’s otter test, which shows how well an AI image maker can make images of an otter sitting on an airplane using wifi”



Maybe you want to see the academic paper about it? “Show me the first page of the academic paper on the Otter test, well-formatted, sitting on a desk” (feel free to zoom in on the text)



Or maybe we should just make it art? “now show an elaborate art gallery, every image on the walls is an otter on an airplane using a laptop, in the styles of Klimt and Rothko and Matisse and Monet and Picasso and Titian and Rembrandt and O’Keefe. There should be readable labels below each one.” (This is worth zooming in on)

All of this is very cool, and would have been impossible a few months ago, but it is useful as well. An image generator that can make detailed text and images can be used to make PowerPoint slides or product mockups or example websites or anything else you ask for. But this is just one tool, and the real magic happens when you combine harnesses, apps, and models on a real problem. Here's one I've been procrastinating about for a decade.

Bringing it together

I am an academic, and a lot of my non-AI work, especially in the early 2010s, focused on crowdfunding. I have hundreds of anonymized data files on the topic that I have collected from surveys and analysis and research work, a mix of STATA, CSV, XLS and Word files that I never got around to writing a paper about. I wanted to see how far GPT-5.5 could get with this information. So, I used Codex powered by GPT-5.5 and asked: “Help me sort [the data] out and generate a new hypothesis that might be interesting and test it in sophisticated ways and write an academic paper.” I also asked it to include a literature review and formatting. The results were very impressive, especially after I asked GPT-5.5 Pro to comment on the paper and fed those results back into Codex. You can read the results here. It isn’t perfect, but that is no longer because there are obvious errors: the literature review is all real, as are the statistics. Instead, it is because, as an expert, I think the hypothesis is not that interesting and there are some standard concerns about causation, even though the AI used very sophisticated statistical methods to try and address them. In short, I would have been very happy if this paper was the outcome of a 2nd year PhD project. And I just gave it four prompts, without ever touching the text myself.

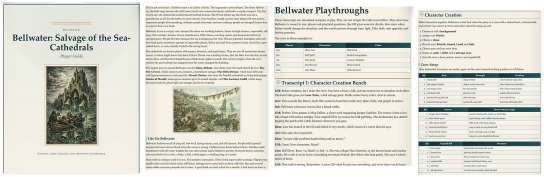
Repairing the Crowd: Creator Communication and the Interpretation of Post-Campaign Failure in Reward Crowdfunding

Abstract

Keywords: crowdfunding; stakeholder theory; trust repair; entrepreneurship platform governance; communication

Introduction

We can bring harnesses and apps and models together another way as well. I asked Codex to create an entirely new tabletop roleplaying game, basically its own version of Dungeons and Dragons in a fantasy world of its own invention, full of all of the tables and rules you need to play. I also asked it to simulate players experiencing the game and revise the rules based on what it found. As you can see, the AI complied, including laying out an attractive 101 page PDF and illustrating it using its image generator.



Bellwater: Salvage of the Sea-Cathedrals

Player Guide

Bellwater Playthroughs

Transcript 1 | Character Creation Reels

In addition to being technically neat, there is a lot to like about the actual content. The setting is interesting and novel, and the rules appear to make sense, drawing on existing game patterns while adding unique elements. However, a closer inspection also reveals the jagged frontier of AI ability is not entirely gone. Every generation of AI models has struggled with actually building long-form fiction. If you are a frequent reader of AI writing you see the same problems here: a love of the uncanny; overly complex ideas that do not fully pay off; weird metaphors ("weather and architecture are the same argument at different speeds"); too many ornate sentences ("the holy things that surface when a sea forgets it was once a road," is cool once, an entire book of that is exhausting); dialogue where every character speaks in the same clipped tone; and the name "Mara." So, even amongst all the amazing technical progress, there are still rough edges.

GPT-5.5 shows us that the models keep getting smarter, the apps keep getting more capable, and the harnesses keep getting better, making them ever more effective at solving real problems. I can get a near PhD-quality paper from four prompts or a playable roleplaying game, illustrated and "playtested," from one. But the fiction is still flat and the hypotheses are sometimes uninteresting even when the statistics are sound. But still. A year ago, none of this was close, and, with the latest releases, capability gains appear to be accelerating.

GPT-5.5 is clearly not the end of this process, but it is a noteworthy step along the way. I have been writing this newsletter for over three years now, and the pattern has not changed: every few months a new model arrives. I run my tests and something that was impossible becomes easy, while the size of the leaps grows each new release cycle. The jagged frontier is still there. It is just much further out than it used to be.

This is how GPT-5.5 chose to illustrate this piece, and who am I to argue?

1 [find in text]

I take no money from OpenAI or any other AI lab, and OpenAI has not seen this post in advance. Also, I don’t know all the details of the launch at the time I am writing this, so I apologize for any errors.